

SOLID Principles (Basic) and Authentication Types

1. SOLID Principles (Basic)

SOLID is a set of five object-oriented design principles that help developers write clean, maintainable, scalable, and reusable code. **S - Single Responsibility Principle (SRP)**
A class should have only one reason to change. It should perform only one responsibility. Example: A Report class should generate reports, while a separate class should save reports to a database.

O - Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. Example: Add new payment methods by creating new classes instead of changing existing code.

L - Liskov Substitution Principle (LSP)

A derived class should be able to replace its base class without affecting program behavior. Example: Any subclass of Bird should behave correctly wherever Bird is used.

I - Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use. Example: Create small, specific interfaces instead of one large interface.

D - Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions, not concrete implementations. Example: A service should depend on an interface rather than a specific database class.

Benefits of SOLID

1. Easier maintenance
2. Better scalability
3. Improved code readability
4. Reduced code duplication
5. Easier testing and debugging

2. Authentication Types

Authentication is the process of verifying the identity of a user, system, or application. **Basic Authentication**

The username and password are sent with every request, usually encoded using Base64. Simple but less secure without HTTPS.

Session-Based Authentication

After login, the server creates a session and stores session information. The client receives a session ID and sends it with future requests.

Token-Based Authentication

The server generates a token after successful login. The client sends the token with each request for verification.

JWT (JSON Web Token)

A compact token containing user information and claims. Widely used in APIs and microservices because it is stateless and scalable.

OAuth 2.0

An authorization framework that allows users to grant limited access to applications without sharing passwords. Examples: Login with Google, GitHub, Facebook, or Microsoft accounts.

OpenID Connect (OIDC)

Built on top of OAuth 2.0 and provides authentication capabilities. Used to verify user identity along with authorization.

SAML (Security Assertion Markup Language)

An XML-based authentication standard commonly used in enterprise Single Sign-On (SSO) systems.

SSO (Single Sign-On)

Allows users to log in once and access multiple applications without logging in again. Common in organizations and enterprises.

Multi-Factor Authentication (MFA)

Requires two or more verification methods such as password, OTP, fingerprint, or authenticator app. Provides stronger security.

Biometric Authentication

Uses fingerprints, facial recognition, iris scans, or voice recognition to verify identity.

API Key Authentication

Applications use a unique API key to access services and APIs.

Comparison

JWT: Best for REST APIs and microservices.

OAuth 2.0: Best for third-party authorization.

OIDC: Best for user authentication and identity management.

SAML: Best for enterprise applications and SSO.

MFA: Best for enhanced security.